

KLERK

6-Day Test Project Sprint Plan

Recruiter-Ready Deliverable at Every Checkpoint

AI Document Processing Assistant MVP

TypeScript • Supabase • Mistral AI • Google Drive • WhatsApp

Day 2 Checkpoint

Working OCR + Extraction Pipeline

→ Recruiter-Worthy

Day 4 Checkpoint

Document Processing + Google Integration

→ Very Impressive

Day 6 Checkpoint

Complete MVP — Deployed & Presented

→ Recruiter-Ready

TABLE OF CONTENTS

1. Project Context & Strategy	Page 3
2. Critical Success Factors	Page 3
3. Day 1: Project Setup + Core Pipeline Foundation	Page 4
4. Day 2: Document Processing Pipeline (Core Business Logic)	Page 5
5. Day 3: Repository Pattern + Database Operations	Page 6
6. Day 4: Google Drive Integration	Page 7
7. Day 5: WhatsApp Simulation + Complete Pipeline	Page 8
8. Day 6: Polish, Test, Deploy, Presentation	Page 9
9. Recruiter Pitch (Interview Talking Points)	Page 11
10. What to Show in GitHub	Page 11
11. Timeline Cheat Sheet (Print This)	Page 12
12. Checkpoints Summary	Page 12
13. Critical Success Rules	Page 13
14. If You Run Out of Time (Priority Order)	Page 13
15. Final Checklist (Day 6, Before Presenting)	Page 14
16. Final Motivation & Trainer Info	Page 14

PROJECT CONTEXT & STRATEGY

Context

You have 6 days to build a working MVP of Klerk that demonstrates:

- ✓ Understanding of AI/ML pipelines (OCR → LLM → extraction)
- ✓ Production-quality code (TypeScript, clean architecture)
- ✓ Real integrations (Supabase, Google, Mistral)
- ✓ Working features (document processing, Google Drive, WhatsApp simulation)

Strategy

- Day 1–2: Build the core document processing pipeline
- Day 3: Repository pattern + database operations
- Day 4: Add Google Drive integration
- Day 5: Add WhatsApp simulation + complete pipeline
- Day 6: Polish, test, deploy, prepare presentation

Result at Each Checkpoint

- Even if you stop at Day 2, you have a working OCR + extraction pipeline.
- Even if you stop at Day 4, you have document processing + Google integration.
- Even if you stop at Day 6, you have a complete MVP to deploy.

CRITICAL SUCCESS FACTORS

1. Code Quality Over Features

- Every commit must have clean, strongly-typed code
- Proper error handling and logging
- Structured like production (not demo code)

2. Demo-Ability

- At the end of each day, you can show: a working feature, clean code, passing tests
- Not theoretical — actually runs

3. Recruiter Narrative

- Build your GitHub README to tell the story:
- "I built Klerk in 6 days, a production-quality document AI assistant."
- Show: Architecture decisions, clean code, working tests, deployment

4. Git Commits (Show Your Work)

- Commit after each task (not at end of day)
- Use conventional commit messages: feat:, fix:, docs:, test:, refactor:
- Each commit should be shippable (tests pass)
- Example: feat: Add Mistral OCR service with confidence scoring

5. README That Tells Your Story

- Problem → Solution → Architecture → How to Run → Demo
- Screenshots of working system
- Link to deployed version

- Code quality highlights

DAY 1: PROJECT SETUP + CORE PIPELINE FOUNDATION

Timeline: 8 hours | Target: Compiling TypeScript with database setup

Morning (4 hours)

TASK 1.1: Initialize Project Structure

 **1 hour**

- mkdir klerk, npm init -y
- Install: typescript, @types/node, ts-node, dotenv, pino, zod
- Create folder structure (see below)

```
klerk/
├── src/
│   ├── config/ (env.ts, logger.ts)
│   ├── domain/ (Document.ts, types.ts)
│   ├── infrastructure/ (database/, external/)
│   ├── services/ (.gitkeep)
│   ├── utils/ (errors.ts, money.ts, retry.ts)
│   └── main.ts
├── tests/
├── tsconfig.json
├── package.json
├── .env.example
└── README.md
```

TASK 1.2: TypeScript Configuration

 **30 min**

- tsconfig.json with strict mode enabled
- Target: ES2020, module: commonjs, strict: true
- esModuleInterop, skipLibCheck, forceConsistentCasingInFileNames
- Include: src, Exclude: node_modules, dist

TASK 1.3: Environment Configuration

 **30 min**

- src/config/env.ts: Zod schema for validation
- Required vars: DATABASE_URL, MISTRAL_API_KEY
- Fails fast if missing — type-safe config export
- .env.example with placeholders only (no secrets)

Afternoon (4 hours)

TASK 1.4: Supabase Database Schema

 **2 hours**

- Create migration: migrations/001_create_documents.sql
- documents table: id (UUID), original_filename, mime_type, sha256_hash (unique)
- document_type, status (default 'pending'), ocr_text, extraction_data (JSONB)
- supplier_name, document_date, due_date, total_ttc (NUMERIC 12,2)
- drive_file_id, drive_web_view_link, created_at, updated_at
- Indexes: sha256_hash, status, document_date

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE TABLE documents (...);
CREATE INDEX idx_documents_hash ON documents(sha256_hash);
```

TASK 1.5: Base Domain Models & Types

🕒 1.5 hours

- src/domain/types.ts: Document interface, ExtractionResult interface, DocumentType enum, DocumentStatus enum
- src/domain/Document.ts: Document class with getters/setters, methods for confidence checking, properly typed

TASK 1.6: Error Handling Foundation

🕒 30 min

- src/utis/errors.ts: KlerkError base class, TransientError, RecoverableError, PermanentError, Global error handler
- src/config/logger.ts: Pino logger setup, structured logging, request IDs

Daily Checkpoint 1 ✓

- ✓ TypeScript compiles without errors
- ✓ npm run build works
- ✓ Database schema created
- ✓ Environment validation works
- ✓ Project structure organized
- ✓ Ready to add services

Commit: *"feat: Project setup with TypeScript, Supabase schema, and error handling"*

Recruiter Narrative:

I set up a production-quality Node.js + TypeScript project with strict TypeScript configuration, Zod environment validation, Supabase PostgreSQL database with proper schema, structured logging with Pino, clean architecture (domain/services/infrastructure separation), and proper error hierarchy for different failure modes.

Time Spent: 8 hours | **Time Remaining:** 40 hours

DAY 2: DOCUMENT PROCESSING PIPELINE (CORE BUSINESS LOGIC)

Timeline: 8 hours | Target: Working OCR + extraction + confidence scoring

Morning (4 hours)

TASK 2.1: Mistral OCR Service

 **1.5 hours**

- npm install @mistralai/mistralai
- src/infrastructure/external/mistral.ts: Initialize Mistral client, handle API key config, error handling
- src/services/OcrService.ts: extractText(file: Buffer) → { text, confidence }
- Call Mistral OCR, return text + confidence, handle timeouts, log all calls

TASK 2.2: Document Classification Service

 **1.5 hours**

- src/services/ClassificationService.ts
- Input: OCR text → Output: DocumentType (invoice | receipt | quote | delivery_note | other)
- Return: { type, confidence } — use Mistral classification

TASK 2.3: Structured Extraction Service

 **1 hour**

- src/services/SupplierInvoiceExtractor.ts
- Extract from OCR text: supplier_name, document_date, due_date, total_ttc (each with { value, confidence })
- Return ExtractionResult with all fields — use Mistral structured extraction
- Parse dates and money correctly

Afternoon (4 hours)

TASK 2.4: Confidence Validation Service

 **1 hour**

- src/services/ConfidenceValidationService.ts
- Check critical fields: supplier_name, total_ttc, document_date, due_date
- Flag if confidence < 0.75
- Return: { isValid, uncertainFields, clarificationPrompt } — prompt in French

TASK 2.5: Duplicate Detection Service

 **1.5 hours**

- src/services/DuplicateDetectionService.ts
- Check SHA256 hash (exact match)
- Check fuzzy match: same doc type + supplier + date + amount ($\pm 0.01\text{€}$)
- Return: { isDuplicate, duplicateDocumentId? } — use DocumentRepository for queries

TASK 2.6: Document Processing Service (Orchestration)

 **1 hour**

- src/services/DocumentService.ts — orchestrate pipeline:
- 1. createDocument(file, filename) → 2. computeHash() → 3. checkDuplicate()
- 4. extractOCR() → 5. classify() → 6. extract() → 7. validateConfidence() → 8. save()

- Return processed document, handle errors gracefully

TASK 2.7: Unit Tests

🕒 30 min

- npm install vitest @vitest/ui
- tests/DocumentService.test.ts: test duplicate detection (hash + fuzzy), confidence validation, extraction parsing
- Mock Mistral API

Daily Checkpoint 2 ✓

- ✓ Document pipeline works end-to-end
- ✓ OCR integration with Mistral
- ✓ Extraction with confidence scores
- ✓ Duplicate detection working
- ✓ Unit tests passing
- ✓ All errors caught and logged

Commit: "feat: Complete document processing pipeline with OCR, extraction, and confidence scoring"

Recruiter Narrative:

I built the core of Klerk — an AI document processing pipeline that receives a document (image/PDF), performs OCR using Mistral, classifies document type automatically, extracts structured fields (supplier, date, amount, etc.), scores confidence for each field (0.0-1.0), flags uncertain fields for user confirmation, detects duplicates (exact hash + fuzzy matching), all properly typed in TypeScript and fully tested with unit tests. The pipeline handles errors gracefully and never invents data. This is production-quality code, not a prototype.

Time Spent: 16 hours total | **Time Remaining:** 32 hours

DAY 3: REPOSITORY PATTERN + DATABASE OPERATIONS

Timeline: 8 hours | Target: Save/retrieve documents, proper data access layer

Morning (4 hours)

TASK 3.1: Repository Base Interface

 **30 min**

- src/repositories/IRepository.ts
- Methods: Save, findById, findByHash, findByDate
- Properly typed generics

TASK 3.2: Document Repository Implementation

 **1.5 hours**

- src/repositories/DocumentRepository.ts — implement IRepository
- Database operations using Supabase client
- Proper error handling, null safety, no hardcoded values

TASK 3.3: Money Utility Functions

 **1 hour**

- src/utils/money.ts
- parseEuro(string): number (cents) — "1 246,80" → 124680
- formatEuro(cents): string — 124680 → "1 246,80 €"
- Never use floating point for money — handle French locale formatting

TASK 3.4: Date Utility Functions

 **1 hour**

- src/utils/dates.ts
- parseDate(string): Date — "12/06/2026" → Date object
- formatDate(date): string — Date → "12/06/2026"
- Handle various French formats

Afternoon (4 hours)

TASK 3.5: Retry Utility with Exponential Backoff

 **1 hour**

- src/utils/retry.ts — withRetry<T>(fn, options)
- Exponential backoff: 500ms → 60s, max 5 attempts
- Pass/fail return, logger integration

TASK 3.6: Integration Tests (Database)

 **1.5 hours**

- tests/integration/DocumentRepository.test.ts
- Save document, retrieve by ID, retrieve by hash, update document
- Test with real Supabase connection

TASK 3.7: Full End-to-End Test

🕒 1 hour

- tests/integration/EndToEnd.test.ts
- Load test image/PDF → process through full pipeline → save to database
- Retrieve and verify → check all fields populated

TASK 3.8: Create Test Fixtures

🕒 30 min

- tests/fixtures/: sample-invoice.pdf (or mock), sample-receipt.png
- Test data factory

Daily Checkpoint 3 ✓

- ✓ Repository pattern implemented
- ✓ Database operations working
- ✓ Money/date utilities correct
- ✓ Retry logic with backoff
- ✓ Integration tests passing
- ✓ End-to-end test passing
- ✓ Can save and retrieve documents

Commit: *"feat: Repository pattern, database operations, and utility functions"*

Recruiter Narrative:

The document pipeline now persists to Supabase: Document Repository implements proper data access layer, money handling uses integer cents (never floats), date parsing handles French locale, retry logic with exponential backoff for resilience, full integration tests with real database, end-to-end tests prove the pipeline works.

Time Spent: 24 hours total | **Time Remaining:** 24 hours

DAY 4: GOOGLE DRIVE INTEGRATION

Timeline: 8 hours | Target: Upload documents to Google Drive, files organized

Morning (4 hours)

TASK 4.1: Google Drive Service Setup

 **2 hours**

- npm install googleapis
- src/infrastructure/external/googleDrive.ts: Service account authentication, initialize Drive client, handle credentials from env
- src/services/GoogleDriveService.ts: uploadFile(file, filename, type) → { fileId, link }
- Create folder structure: Compta/Year/Month/DocumentType/
- Cache folder IDs to avoid recreating, return Drive file ID and shareable link

TASK 4.2: Google Drive File Upload Worker

 **1.5 hours**

- src/workers/GoogleDriveUploadWorker.ts
- Queue job after document extraction
- Download document from buffer → upload to Drive
- Update document with drive_file_id, handle quota errors gracefully

TASK 4.3: Create Simulated Google Drive (for testing)

 **30 min**

- If you can't get Google credentials working:
- Create mock GoogleDriveService, store files locally in /uploads
- Return mock file IDs — still shows the integration pattern

Afternoon (4 hours)

TASK 4.4: Google Sheets Integration

 **2 hours**

- src/services/GoogleSheetsService.ts
- Authenticate with Google Sheets API, append row to journal
- Format: Date | Supplier | Type | Amount | Drive Link | Status
- Create journal if not exists

TASK 4.5: Google Sheets Append Worker

 **1 hour**

- src/workers/GoogleSheetsAppendWorker.ts
- Queue job after Drive upload, append formatted row to Sheets
- Handle failures with retry

TASK 4.6: Integration Tests

 **1 hour**

- tests/integration/GoogleIntegration.test.ts
- Mock Google API (or use test account)
- Test folder creation, file upload, Sheets append, verify data in Sheets

Daily Checkpoint 4 ✓

- ✓ Google Drive integration working (real or mocked)
- ✓ Files upload to Drive
- ✓ Folder structure created correctly
- ✓ Google Sheets journal populated
- ✓ All integration tests passing
- ✓ Full pipeline: Document → OCR → Extract → Drive → Sheets

Commit: *"feat: Google Drive and Sheets integration with file organization"*

Recruiter Narrative:

The pipeline now integrates with Google services: automatically uploads documents to Google Drive, organizes files in folder structure (Compta/Year/Month/Type/), appends entry to Google Sheets accounting journal, tracks all documents with drive links and metadata, handles both real and simulated Google APIs for testing, production-ready error handling.

Time Spent: 32 hours total | **Time Remaining:** 16 hours

DAY 5: WHATSAPP SIMULATION + COMPLETE PIPELINE

Timeline: 8 hours | Target: Simulate WhatsApp message receiving, complete working MVP

Morning (4 hours)

TASK 5.1: WhatsApp Message Handler (Simulated)

 **2 hours**

- src/api/whatsapp.ts: Create POST /api/whatsapp/webhook endpoint
- Accept file upload (simulate WhatsApp media), queue document processing
- Send simulated response
- Note: For true WhatsApp integration, you'd need Unipile API. For now, simulate by accepting file uploads and returning JSON responses.

TASK 5.2: Express API Server

 **1.5 hours**

- npm install express
- src/main.ts: Initialize Express app
- Mount routes: /api/whatsapp, /api/health, /api/documents
- Error middleware, health check: GET /health → { status: 'ok' }
- Readiness: GET /ready → { ready: boolean }

TASK 5.3: WhatsApp Response Service

 **30 min**

- src/services/WhatsappResponseService.ts — format French messages:
- Confirmation: "Document reçu. Traitement en cours..."
- Clarification: "Je lis: [fields]. Confirmez-vous?"
- Error: "Format non supporté. Envoyez JPG ou PDF."
- Return as JSON (for simulated endpoint)

Afternoon (4 hours)

TASK 5.4: Job Queue Simulation (pg-boss)

 **1.5 hours**

- npm install pg-boss
- src/infrastructure/queue/pgBossQueue.ts: Initialize pg-boss
- Queue names: document.ingest, document.ocr, document.extract, etc.
- Job handlers that orchestrate services

TASK 5.5: Queue Workers

 **1.5 hours**

- src/workers/index.ts: Start job processors
- Handle document.ingest (receives document)
- Handle document.process (orchestrate pipeline)
- Handle google.upload, Handle notification.whatsapp (send message back)

TASK 5.6: End-to-End Demo Script

🕒 1 hour

- scripts/demo.ts: Load sample document → send to /api/whatsapp endpoint
- Wait for processing → retrieve from database
- Show: Original file, OCR text, extracted data, confidence scores
- Show: Duplicates detected, Google Drive link, Sheets entry

Daily Checkpoint 5 ✓

- ✓ Express API running on localhost:3000
- ✓ WhatsApp webhook accepts file uploads
- ✓ Full pipeline works via queue
- ✓ Demo script runs successfully
- ✓ All integration tests passing
- ✓ Complete MVP working end-to-end

Commit: *"feat: WhatsApp API simulation, job queue, and complete working MVP"*

Recruiter Narrative:

The MVP is now complete and working: HTTP API endpoint simulates WhatsApp webhook, job queue processes documents asynchronously, full pipeline (Document → OCR → Extraction → Confidence → Duplicate Check → Google Drive → Sheets), demo script shows the entire flow working, all components properly typed and tested, can handle multiple documents in parallel, graceful error handling at every step.

Time Spent: 40 hours total | **Time Remaining:** 8 hours

DAY 6: POLISH, TEST, DEPLOY, PRESENTATION

Timeline: 8 hours | Target: Deploy on Render, create compelling GitHub README, prepare presentation

Morning (4 hours)

TASK 6.1: Docker Setup

🕒 1 hour

- Dockerfile: Multi-stage build, Node.js 20 Alpine, optimized for free tier
- docker-compose.yml: PostgreSQL with pgvector, Node.js app, all dependencies included

TASK 6.2: Test Coverage & Validation

🕒 1 hour

- Run: npm run test, npm run build, npm run lint
- Verify: All tests passing, TypeScript no errors, Zero console warnings

TASK 6.3: GitHub Actions CI

🕒 1 hour

- .github/workflows/ci.yml: On every push — lint, type-check, test, build
- Must pass before merge

TASK 6.4: Professional README

🕒 1 hour

- README.md is what recruiters read — include:
- Problem statement, What It Does (8-step pipeline), Architecture diagram
- Tech Stack table, Features Implemented checklist, Performance metrics
- Example outputs (invoice processing, duplicate detection, low confidence)
- Code Quality highlights, How to Run Locally, API Endpoints
- Production Deployment, What I'd Build Next, Learnings from Sprint

Afternoon (4 hours)

TASK 6.5: Prepare Deployment

🕒 1.5 hours

- Create Render deployment: Connect GitHub repo, set environment variables
- Deploy backend, set /health endpoint for monitoring

TASK 6.6: Create Architecture Diagram

🕒 1 hour

- Use Mermaid or simple ASCII diagram showing the full pipeline:
- WhatsApp/Gmail/API → Document Ingest → OCR (Mistral) → Classification → Extraction
- → Confidence Validation → Duplicate Detection → Google Drive + Sheets → Response to User
- Save as: docs/ARCHITECTURE.md

TASK 6.7: Create Demo Video / Screenshots

🕒 1 hour

- Show: Upload document via API, watch processing in logs, view extracted data in database

- Show: Google Drive file created, Sheets entry created, confidence scores, duplicate detection
- If time: Screen recording (1-2 min). If not: Screenshots with annotations

TASK 6.8: Final Polish & Testing

 30 min

- Checklist: No TypeScript errors, all tests passing, no console.logs (use logger)
- Comments on complex logic, README complete and accurate, GitHub repo public
- Deployed on Render, demo script works, git history clean

Daily Checkpoint 6 ✓

- ✓ Deployed on Render (live URL)
- ✓ All tests passing
- ✓ Professional README
- ✓ Architecture documented
- ✓ Demo working
- ✓ GitHub public
- ✓ Ready for recruiter review

Commit: *"docs: Complete README, architecture, and deployment documentation"*

RECRUITER PITCH (What You Say at the Interview)

RECRUITER PITCH (What You Say at the Interview)

"I built Klerk in 6 days as a test project. It's an AI document processing assistant that:"

1. Solves a Real Problem

- French tradespeople spend 30+ minutes/day on accounting data entry
- Klerk automates extracting and filing documents

2. Demonstrates Full Stack

- Backend: Node.js + TypeScript + PostgreSQL
- AI/ML: Mistral OCR, classification, structured extraction
- Integrations: Google Drive, Google Sheets
- Queue: Asynchronous processing with exponential backoff
- Testing: Unit + integration tests with real data

3. Production-Quality Code

- Strict TypeScript (no any types)
- Proper error handling (never invent data)
- Confidence scoring (tells user when uncertain)
- Duplicate detection (prevents re-processing)
- Structured logging (troubleshooting)
- Clean architecture (easy to extend)

4. Handles Edge Cases

- Blurry documents → asks user for confirmation
- Rate limits → exponential backoff
- Duplicate files → detects and skips
- Network errors → retries transparently
- Large files → streams, never loads into memory

5. Actually Works

- Can process documents end-to-end
- Files upload to Google Drive
- Data logged in Sheets
- Deployed and live

6. Shows Growth

- Day 1–2: Core pipeline (foundation)
- Day 3: Database layer (reliability)
- Day 4: Google integration (real service)
- Day 5: API + Queue (scalability)
- Day 6: Deployment (production)

"The code is on GitHub. You can see every step. It's not theoretical — it's shipping code that handles real requirements. What I'd build next: Real WhatsApp integration, vector search, dashboard, monthly reports. But the foundation is solid and extensible."

Key Points to Emphasize

- ✓ Correct over clever (no hallucinations, never invent data)
- ✓ Production code from day 1 (strict TypeScript, testing)

- ✓ Integration with real services (Mistral, Google, Supabase)
- ✓ Proper architecture (can add features without rewriting)
- ✓ Error handling (what happens when things fail?)
- ✓ Testability (can prove it works)
- ✓ Actually deployed (not just local)

WHAT TO SHOW IN GITHUB

Critical Files to Commit

File	Why It Matters
<code>src/services/DocumentService.ts</code>	Core orchestration — shows you understand pipelines
<code>src/services/OcrService.ts</code>	Mistral integration, confidence scoring, error handling
<code>src/services/DuplicateDetectionService.ts</code>	Business logic — shows thinking about edge cases
<code>src/repositories/DocumentRepository.ts</code>	Data access layer — shows pattern knowledge
<code>tests/integration/EndToEnd.test.ts</code>	Proves it works — shows testing approach
<code>README.md</code>	Tells the story — shows professionalism
<code>.github/workflows/ci.yml</code>	Shows DevOps thinking — automated testing/building

Don't Commit

✗ `node_modules/` ✗ `.env` (put vars in GitHub Secrets) ✗ `dist/` (build artifacts) ✗ `console.logs` or debugging code

Use a `.gitignore`:

```
node_modules/
dist/
.env
.env.local
coverage/
*.log
```

TIMELINE CHEAT SHEET (Print This)

Day 1 (8 hours): PROJECT SETUP

- ✓ TypeScript project initialized
- ✓ Supabase schema created
- ✓ Environment validation
- ✓ Error hierarchy

Commit: *"feat: Project setup"*

Day 2 (8 hours): OCR + EXTRACTION PIPELINE

- ✓ Mistral OCR service
- ✓ Classification
- ✓ Extraction with confidence
- ✓ Duplicate detection
- ✓ Orchestration
- ✓ Unit tests

Commit: *"feat: Document processing pipeline"*

Day 3 (8 hours): DATABASE LAYER

- ✓ Repository pattern
- ✓ Save/retrieve documents
- ✓ Money utilities (no floats)
- ✓ Date utilities
- ✓ Retry logic
- ✓ Integration tests

Commit: *"feat: Repository pattern and utilities"*

Day 4 (8 hours): GOOGLE INTEGRATION

- ✓ Google Drive upload
- ✓ Folder organization
- ✓ Google Sheets journaling
- ✓ Mocked or real integration

Commit: *"feat: Google Drive and Sheets integration"*

Day 5 (8 hours): API + QUEUE

- ✓ Express API
- ✓ WhatsApp endpoint (simulated)
- ✓ pg-boss job queue
- ✓ Workers for each stage
- ✓ End-to-end demo

Commit: *"feat: API and job queue"*

Day 6 (8 hours): DEPLOYMENT + PRESENTATION

- ✓ Docker setup
- ✓ Tests passing
- ✓ Professional README
- ✓ Deployed on Render

✓ GitHub public

✓ Demo working

Commit: *"docs: README and deployment"*

CHECKPOINTS SUMMARY

Minimum Viable (Day 2 end)

You have: Working document processing pipeline

You can show: OCR + extraction + confidence scoring — Production-grade

→ **THIS ALONE IS IMPRESSIVE**

Solid (Day 4 end)

You have: Document processing + Google integration + database

You can show: Full pipeline with persistent storage — Production-grade, tested

→ **VERY IMPRESSIVE**

Complete (Day 6 end)

You have: Full MVP with API, queue, deployment

You can show: Complete working system — Production-grade, tested, deployed

→ **RECRUITER-READY**

Even if you can only finish Day 3–4, you have something substantial to show.

CRITICAL SUCCESS RULES

1. Every Commit is Shippable

- Never commit broken code
- Tests pass before commit
- Clean code always

2. No Fake Data

- Never invent confidence scores
- Never pretend extraction worked if it didn't
- Transparency in errors

3. Production Code, Not Demo Code

- Use TypeScript strict mode
- Error handling everywhere
- Logging at important points
- Configuration over constants

4. Tests Prove It Works

- Unit tests for services
- Integration tests for pipeline
- At least one end-to-end test
- Run tests before committing

5. Documentation is Code

- README that sells your work
- Code comments on complex logic
- Architecture diagrams
- API documentation

6. Deploy Early

- Day 5–6 have working code
- Get it live (even if incomplete)
- Show recruiters a URL they can visit
- "It's deployed here: <https://klerk.render.com>"

IF YOU RUN OUT OF TIME

Priority order — implement in this order:

P0 (Critical)

- ✓ Document upload works
- ✓ OCR + extraction working
- ✓ Database saves data
- ✓ Code is properly typed (TypeScript)

P1 (High)

- ✓ Google Drive integration (or mock)
- ✓ Confidence scoring shown
- ✓ Duplicate detection working

- ✓ Tests passing

P2 (Medium)

- ✓ Job queue working
- ✓ API endpoints
- ✓ Demo script
- ✓ Professional README

P3 (Nice-to-Have)

- ✓ Deployed on Render
- ✓ Google Sheets integration
- ✓ WhatsApp simulation
- ✓ Multiple document types

If you run out of time:

- Stop at P0 or P1
- Focus on quality (not features)
- Document what you built
- Explain what comes next
- Show recruiters clean code

Remember: 1 fully working feature > 5 partially working features

FINAL CHECKLIST (Day 6, Before Presenting)

Code

- ☐ TypeScript strict mode enabled
- ☐ No TypeScript errors (npm run build)
- ☐ ESLint passes (npm run lint)
- ☐ All tests pass (npm run test)
- ☐ No console.logs (use logger)
- ☐ No hardcoded values
- ☐ Proper error handling everywhere

Documentation

- ☐ README complete and accurate
- ☐ Architecture diagram included
- ☐ Code comments on complex logic
- ☐ API documentation
- ☐ How to run locally
- ☐ How to deploy

Testing

- ☐ Unit tests for services
- ☐ Integration tests working
- ☐ End-to-end demo runs
- ☐ Handles edge cases

Deployment

- ☐ Docker builds successfully
- ☐ Deployed on Render (or local demo)
- ☐ Health check endpoint works
- ☐ Can show working system

Git

- ☐ Clean commit history
- ☐ Good commit messages
- ☐ No build artifacts committed
- ☐ .env not committed

Presentation

- ☐ Can explain architecture
- ☐ Can show code (specific examples)
- ☐ Can run demo
- ☐ Can discuss design decisions
- ☐ Can explain what comes next

YOU'VE GOT THIS 🚀

6 days is not much time, but it's enough to build something impressive if you:

- Stay focused (follow the timeline)
- Prioritize quality over features
- Commit working code regularly

- Document as you go
- Test constantly

At the end of Day 2, you have something recruiter-worthy.

At the end of Day 4, you have something impressive.

At the end of Day 6, you have a complete MVP.

Stick to the plan. Don't try to do everything perfectly.

Done and working > perfect and incomplete.

You're building a test project to get on the real team.

Show them you can write production code, not prototype code.

Show them you understand architecture, not just coding.

Show them you can ship features, not just talk about them.
